EmbedEval - Benchmarking Large Language Models for Embedded Software Engineering Tasks

Rishi Mantri
Department of Electrical and Computer Engineering
Purdue University
West Lafayette, Indiana, USA
mantrir@purdue.edu

Devesh Maheshwari
Department of Electrical and Computer Engineering
Purdue University
West Lafayette, Indiana, USA
maheshwd@purdue.edu

Ayush Bansal
Department of Computer Science
Purdue University
West Lafayette, Indiana, USA
bansa162@purdue.edu

Aanya Mittal
Department of Electrical and Computer Engineering
Purdue University
West Lafayette, Indiana, USA
mitta122@purdue.edu

Alan Hsi
Department of Computer Science
Purdue University
West Lafayette, Indiana, USA
hsi0@purdue.edu

***Abstract*—Large language models demonstrate strong performance in code generation across various domains, as shown by general software engineering benchmarks. However, their ability to generate code for real-world embedded software remains poorly understood. Embedded software requires reasoning about hardware-specific constraints and platform-level behavior - capabilities that general benchmarks do not adequately assess. Given that embedded failures can have direct physical consequences and debugging is constrained by expensive hardware-in-the-loop cycles, understanding where LLMs succeed and fail in this domain is especially important.**

We present a benchmarking framework for evaluating LLMs on professional embedded software tasks derived from real pull requests across popular embedded repositories. Our benchmark covers task categories that reflect real engineering workflows like debugging, feature development, and driver configuration. In addition, it integrates with hardware simulation environments to enable fully reproducible evaluation without physical hardware. We will use this framework to evaluate several frontier LLMs, identifying where current models consistently struggle on embedded tasks and establishing a baseline for future work in this domain.

I. Introduction

Embedded systems form a fundamental layer of modern computing architecture and are widely used in various domains, including medical devices, automated control systems, and networked communication hardware. The global embedded software market was valued at approximately $17.91 billion in 2024 and is projected to exceed $30.23 billion by 2030 [1] . Similarly, the broader embedded systems market was valued at approximately $112.3 billion in 2024 and is expected to reach $169.1 billion by 2030 [2]. These figures highlight the scale and growing importance of this field.

Unlike general-purpose software, embedded software operates under strict hardware constraints, including limited memory resources, real-time scheduling requirements, and direct interaction with peripherals through device drivers and hardware abstraction layers. Consequently, defects in embedded software can lead to failures with direct physical consequences. Furthermore, the cost of validation and debugging is significantly higher than in conventional software development due to the reliance on hardware-in-the-loop testing and specialized simulation infrastructure.

These challenges create a compelling need for tools that can improve developer productivity and reduce development costs in this domain. Large language models have recently emerged as a promising direction toward this goal, demonstrating strong capability across a broad range of programming tasks. However, their ability to handle the hardware-specific constraints and platform-level reasoning that embedded software demands remains poorly understood, and this gap is what this research directly addresses.

II. Background and Related Works

This section reviews prior work that motivates the design of EmbedEval. We first discuss general software engineering benchmarks for large language models, highlighting the evaluation methodologies they establish and the limitations they present for embedded development. We then examine emerging work on the use and evaluation of LLMs in embedded and hardware-aware software tasks. Finally, we review embedded software testing and simulation infrastructure, focusing on execution environments that enable scalable and reproducible validation without requiring physical hardware.

A. General SWE Benchmarks

The evaluation of LLMs on coding tasks has progressed through several stages of increasing complexity, beginning with function-level code generation. Early benchmarks such as HumanEval [3] and MBPP [4] established the standard evaluation paradigm: given a natural language description, the model generates a candidate solution, and correctness is measured using the pass@k metric, where a task is considered solved if any of k generated samples passes all

TABLE I: Comparison of EmbedEval with prior benchmarks discussed in related work.

Benchmark / Work	Real Repo Tasks	Agentic Repair	Multi-file Context	Embedded / RTOS Code	Executable Validation
HumanEval / MBPP / EvalPlus	No	No	No	No	Unit tests
CodeContests / LiveCodeBench	No	No	No	No	Unit tests
CrossCodeEval	Partial	No	Yes	No	No
SWE-Bench / OmniCode	Yes	Yes	Yes	No	Unit tests
Englhardt et al.	No	No	Limited	Yes	Hardware-in-loop
EmbedAgent / EmbedBench	No	Partial	Limited	Partial	Virtual hardware
EmbedEval	**Yes**	**Yes**	**Yes**	**Yes**	**Simulator / tests**

test cases. HumanEval consists of 164 hand-crafted Python problems, while MBPP contributes 974 crowd-sourced tasks at a more basic difficulty level. However, subsequent work through EvalPlus [5] demonstrated that the test suites in both benchmarks were insufficient for reliably measuring correctness. By augmenting them with 80× and 35× more test cases using LLM-generated seed inputs and type-aware mutation, EvalPlus revealed pass@k reductions of up to 19.3% to 28.9% across 26 evaluated models, showing that many previously accepted solutions were in fact incorrect.

To evaluate deeper algorithmic reasoning, benchmarks like CodeContests [6] collected over 13,000 competitive programming problems from platforms such as Codeforces and AtCoder, using temporal data splits to prevent leakage and generated tests to reduce false positives. At the repository level, CrossCodeEval [7] tested cross-file code completion across four languages, using static analysis to ensure that each task strictly requires context from other files and cannot be solved from the current file alone. LiveCodeBench [8] addressed a different concern, data contamination, by continuously collecting new problems from weekly programming contests with release-date tagging, enabling temporal evaluation of whether model performance degrades on problems published after training.

While these benchmarks primarily evaluate code generation ability, a separate line of work has focused on evaluating LLMs as software engineering agents. SWE-Bench [9] shifted the focus towards real software engineering by tasking LLMs with resolving actual GitHub issues from popular open-source Python repositories. Each task instance consists of a repository snapshot at a specific commit, a natural language description derived from a real issue or pull request, and a set of tests that validate the fix, requiring to reason about the codebase, and produce a correct patch. This methodology of grounding evaluation in real developer workflows and existing repositories makes SWE-Bench a particularly compelling framework for assessing practical engineering capability. OmniCode [10] further extends this direction by proposing a benchmark for evaluating full software engineering agents. Together, these benchmarks have significantly advanced the evaluation of LLMs for general-purpose software tasks, yet they share a critical limitation: they focus exclusively on high-level programming languages, predominantly Python. None of the aforementioned benchmarks evaluate LLM performance on embedded software engineering, which requires reasoning about hardware-specific constraints, peripheral configuration, memory-mapped I/O, and real-time behavior. Furthermore, no existing benchmark integrates hardware simulation environments for reproducible evaluation. This gap motivates our work on EmbedEval, which adapts the real-world, repository-grounded methodology established by SWE-Bench to the embedded software domain.

B. LLMs for Hardware/Embedded Software

LLMs are increasingly being explored in embedded software development for tasks that require hardware awareness and platform-specific reasoning.

Prior work has explored the use of LLMs for automated generation of hardware-specific software components, such as HAL code for STM32 microcontrollers, showing that LLM-based methods can assist with low-level embedded code generation and integration into existing codebases [11]. LLMs have also been studied for embedded IoT code migration across operating systems, highlighting their potential in tasks that require adapting software across platform-specific APIs and embedded environments [12]. In addition, recent work has investigated natural-language-based generation of embedded IoT applications across platforms such as RIOT, Zephyr, Contiki, and FreeRTOS, further demonstrating the promise of LLMs in embedded development workflows [13]. Together, these works show that LLMs are no longer being considered only for general-purpose programming, but are increasingly being applied to embedded software tasks involving hardware abstraction, OS-specific code, platform interfaces, and peripheral control.

Despite this growing interest, systematic evaluation in the embedded domain remains limited. Unlike general software engineering, embedded systems still lack broadly adopted benchmarking frameworks and standardized evaluation settings for comparing model performance. This motivates a closer look at the small body of work that has begun to evaluate LLMs more explicitly in embedded contexts.

1) Early Benchmarking Efforts for Embedded LLM Evaluation: An important early direction in this area is the work of Englhardt et al., which evaluates LLMs directly on executable embedded tasks. As shown in Fig. 1, this approach uses a hardware-in-the-loop evaluation framework in which model-generated code is compiled, uploaded to

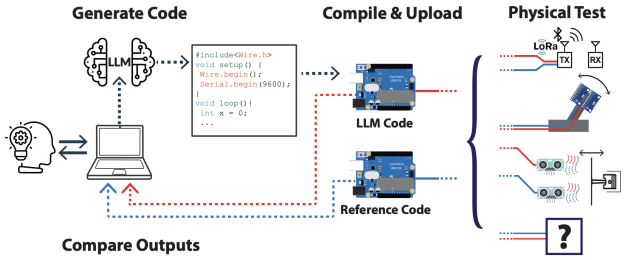

Fig. 1: Hardware-in-the-loop evaluation for embedded LLM assessment [14].

microcontrollers, and physically validated against reference implementations using sensor-actuator setups. This line of work studies whether LLMs can support end-to-end embedded development rather than only producing syntactically correct code, and evaluates models such as GPT-3.5, GPT-4, and PaLM 2 across real hardware tasks involving peripherals such as photoresistors, ultrasonic sensors, and IMUs [14]. This is especially significant because it demonstrates that embedded LLM evaluation can move beyond static inspection and into executable, hardware-aware validation.

A more benchmark-oriented direction is represented by EmbedAgent, which moves from exploratory characterization toward a more explicit and automated benchmark design. In this line of work, embedded evaluation is framed through multiple developer roles, including writing code from a task description and schematic, jointly designing circuits and code, and migrating designs across platforms. The resulting benchmark, EmbedBench in Fig. 2, evaluates tasks spanning embedded programming, circuit design, and cross-platform migration across Arduino Uno, ESP32, and Raspberry Pi Pico, using the Wokwi hardware emulator / virtual platform to execute and validate generated behavior automatically [15]. Compared to earlier work, this approach is closer to a true benchmark setting because it introduces broader task structure, automated validation, and more reproducible evaluation through virtual execution.

2) Remaining Gaps in Prior Work: Although prior work has made important progress, existing benchmarks do not fully capture the embedded software maintenance workflow targeted by EmbedEval. In this work, we focus on real RTOS pull requests involving bug fixes, feature additions, refactors, and API consistency changes. These tasks require models to modify existing codebases, work across multiple files, interact with project build systems, and reason about embedded subsystems such as scheduling, POSIX layers, networking, logging, drivers, and configuration infrastructure.

General code-generation benchmarks such as HumanEval, MBPP, and related test-based benchmarks evaluate self-contained programming tasks. They are useful for measuring function-level synthesis, but they do not require repository navigation, multi-file edits, build-system interaction, or iterative debugging. Repository-level benchmarks such as SWE-Bench move closer to real software engineering by using real issues and test-based validation, but they focus on general-purpose software rather than embedded RTOS code. As a result, they do not directly evaluate cross-compilation, RTOS APIs, kernel and driver code, Kconfig/devicetree-style configuration, or simulator-based embedded testing.

Existing embedded LLM evaluations address parts of this gap by using hardware or virtual execution environments, but they often focus on generated programs from task specifications rather than real upstream PR repair. EmbedEval therefore targets the intersection that remains underexplored: real embedded repository tasks, iterative agentic repair, multi-file RTOS code changes, and executable validation through the project's own simulation-supported test infrastructure.

C. Embedded Software Testing and Simulation Infrastructure

Embedded software is harder to validate than conventional software because correctness depends not only on source code logic, but also on the underlying board, architecture, peripherals, and execution environment. In many cases, compile-only checks are not sufficient, while direct testing on physical hardware can be slow, expensive, and difficult to reproduce consistently across repeated runs. For this reason, prior work increasingly motivates the use of virtual embedded execution environments as a practical alternative for development and validation workflows [16], [11]. In particular, these environments are useful because they can:

- reduce dependence on repeated physical prototyping,
- support controlled and repeatable testing,
- execute real embedded software rather than only abstract models, and
- integrate naturally into automated validation pipelines [16], [11], [17], [18], [19].

1) QEMU in Realistic Embedded and IoT Workflows: Prior work shows that ISA-based emulation with QEMU can support realistic embedded and IoT evaluation workflows, especially when the goal is to run real software stacks rather than simplified synthetic models. Schirrmeister et al. present a simulation and benchmarking environment for IoT device usage scenarios using Zephyr and QEMU, where QEMU-virtualized embedded devices run Zephyr-based applications in order to evaluate a device command and control infrastructure under realistic load conditions [17]. A key motivation in that work is that traditional network simulators such as ns-3 or OMNeT++ would require separate models of the system behavior, whereas their approach instead executes the real application software on virtualized devices [17]. This is especially relevant for embedded benchmarking because it shows that emulator-based execution can preserve realistic software behavior while still enabling automation and repeatability. A related perspective appears in IoTNetEMU, which is motivated by the need to emulate a significant number of nodes running real code and connecting to application services through a controllable network [18]. Although its

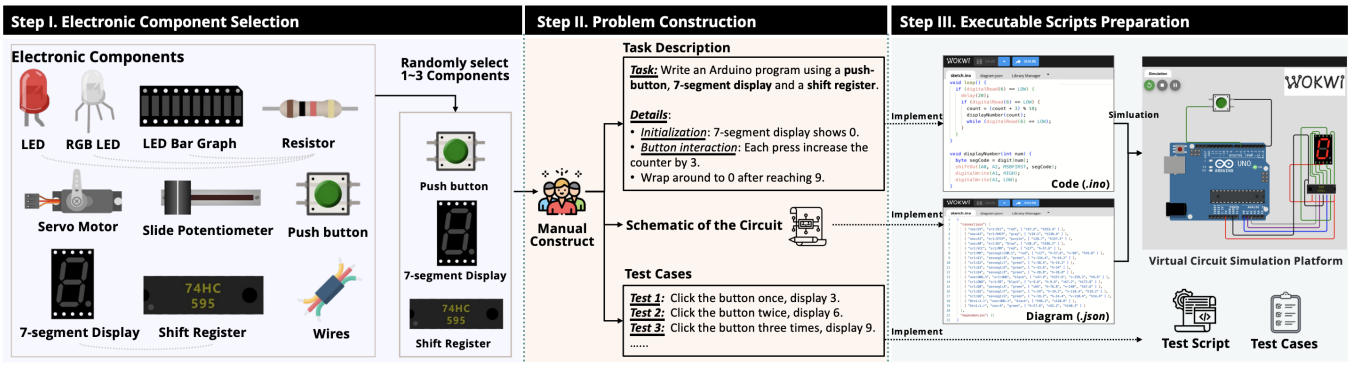

Fig. 2: Automated benchmark structure for embedded programming, circuit design, and migration tasks [15].

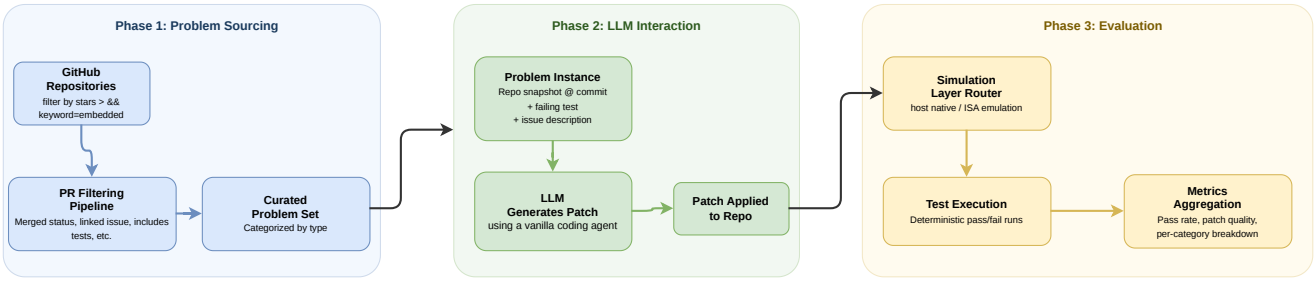

Fig. 3: End-to-end evaluation pipeline for EmbedEval

scope is broader than QEMU alone, the framework likewise reinforces the value of virtual execution for testing IoT applications in settings that are both more realistic than pure modeling and more practical than large physical testbeds [18].

2) Renode for Higher-Fidelity Hardware-Aware Validation: Compared to more CPU- and platform-oriented emulation workflows, the Renode literature emphasizes hardware-aware validation with stronger support for board-level structure and peripheral interaction. Speiser et al. describe Renode as an open-source embedded system simulation framework for software-based testing and virtual functional validation, arguing that simulating hardware and software together can reduce the cost and time associated with repeated prototyping and hardware verification [16]. Haug et al. likewise use Renode as the validation backend for automatically generated STM32F407 HAL code, where generated code is compiled and executed in a simulated environment to verify correctness without requiring physical hardware [11]. Their use of Renode is notable because it is tied directly to microcontroller-specific registers and HAL behavior rather than only higher-level application execution [11]. More recently, Renode has also been explored in the context of digital-twin construction, including a case study involving an STM32L4 platform and an AT86RF233 ZigBee radio [20]. Together, these works suggest that Renode-style hardware-aware emulation is especially useful when evaluation requires greater fidelity to platform structure, peripheral behavior, and firmware-level interaction.

III. Methodology

EmbedEval aims to evaluate LLMs in real-world embedded software engineering tasks. The key components of our methodology for doing so include problem sourcing, LLM-driven code generation, and simulation-based evaluation. Fig. 3 illustrates these components. The framework will include a set of curated problems derived from real pull requests in popular embedded repositories. Each problem instance consists of a repository snapshot at a specific commit, a natural language task description derived from the original issue or PR, and a set of tests. An LLM is provided with the relevant context and is tasked with generating the code to resolve the issue. The generated code is applied to the repository and the appropriate simulation layer is used for test execution, producing a deterministic pass/fail result. This pipeline design is inspired by the methodology established by SWE-Bench [9], which evaluates LLMs on real GitHub issues from Python repositories but adapted for the constraints and problem types in embedded software.

A. Simulation architecture

A central design goal of the framework is to enable fully reproducible evaluation without requiring physical hardware. Embedded software testing commonly relies on hardware-in-the-loop (HIL) setups, which are expensive, difficult to scale, and inherently non-deterministic across test runs [21]. To address this, we plan to use a simulation-based approach that leverages various layers of execution environments, each with a different tradeoff between execution speed and hardware

fidelity. Fig. 4 illustrates this architecture and maps each layer to the categories of embedded problems it is best suited to evaluate.

1) Host-native execution: This is the fastest and least hardware-faithful layer, where the embedded application is compiled against POSIX-compatible abstractions and executed as a native Linux binary on the host machine. For example, in the Zephyr ecosystem, this is implemented as the native-sim board target, which replaces hardware abstractions with host-side POSIX stubs [22]. FreeRTOS provides an analogous capability through its POSIX/Linux simulator port, which emulates task scheduling on the host [23]. This layer is only appropriate for testing application-level code and RTOS logic, including task scheduling correctness or data structure integrity. Since the code under test is decoupled from the underlying hardware, the advantage for benchmarking is access to host compilation and debugging tools at minimal overhead. However, this layer requires explicit framework support as the target codebase must provide a simulation target for the host. This is available in several major RTOS projects (Zephyr, FreeRTOS, RIOT, NuttX) but may not be in vendor SDKs, bare-metal firmware, or other repositories that interact directly with hardware registers. For such codebases, evaluation begins at the emulation layer.

2) ISA-level emulation: At this layer, firmware compiled for the target architecture is executed within an emulator that models the target CPU, memory map, and varying degrees of peripheral behavior by performing binary translation of instructions on the host. The firmware binary at this layer is identical to what would be flashed onto physical hardware, providing substantially higher fidelity than host-native execution. This allows for testing problems involving ISA-dependent behavior, memory-mapped I/O, and peripheral driver logic. Unlike the host-native layer, it can emulate hardware dependent firmware, making it the universal baseline for our framework. We plan to use QEMU [24] and Renode [25] as the primary emulation tools in this layer. QEMU is a general machine emulator with support for various processors and broad platform integrations with projects like Zephyr. While it benefits from a mature community, it has limited support for peripherals and struggles with reproducibility in timing-sensitive tests. Renode is a framework which provides hardware simulation specifically for embedded systems development. It includes better platform-level modeling (SPI, I²C, and CAN implementations) along with more deterministic execution [16]. Within our framework, problems will be routed to whichever emulator provides adequate hardware fidelity for the specific test case. In some projects like Zephyr, tests specify the platforms they are allowed to run on, and this metadata can be used to guide routing automatically. For other repositories, the appropriate environment will need to be determined through manual analysis of the problem's hardware dependencies.

3) Cycle-accurate simulation: Both host-native execution and system-level emulation abstract away the precise timing behavior of the target processor. And so neither can faithfully

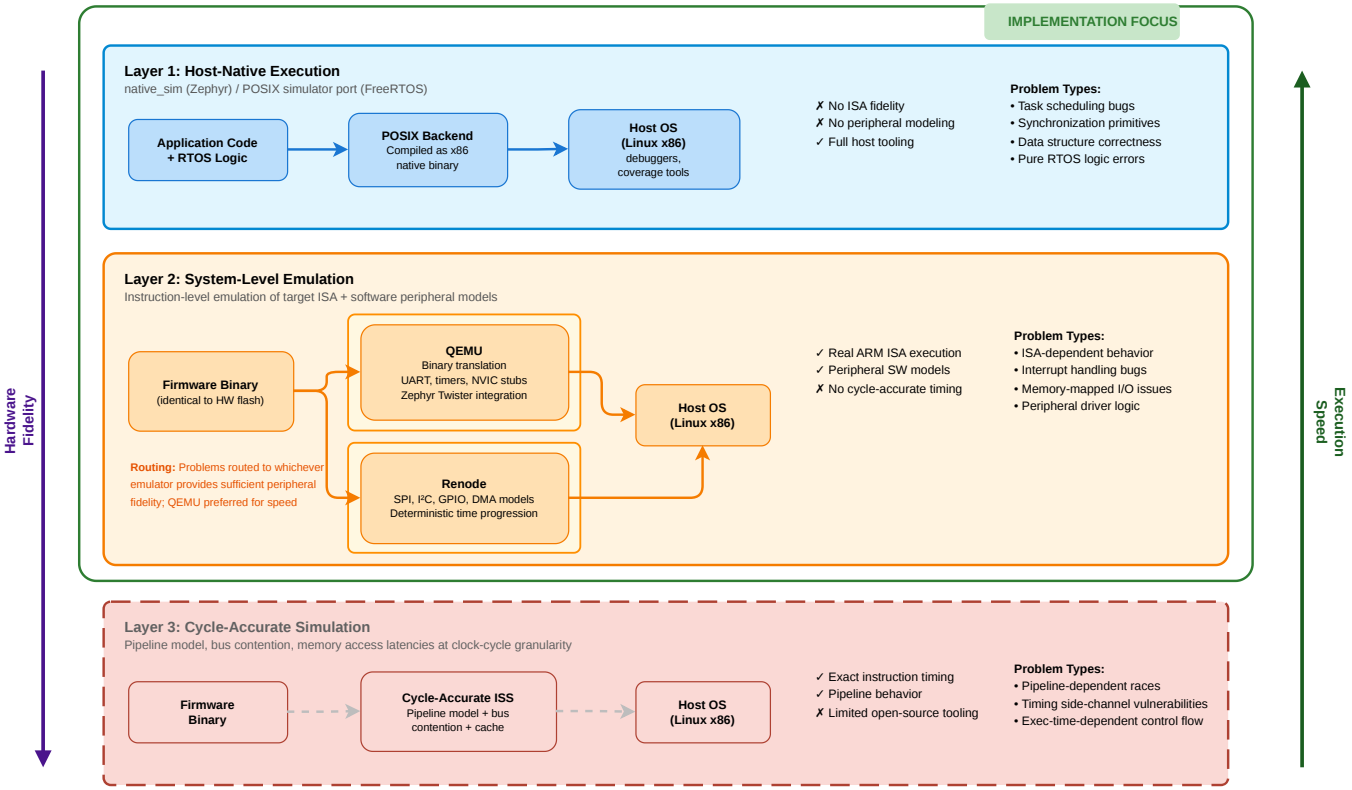

Fig. 4: Simulation Layer Architecture

reproduce bugs that depend on exact instruction timing such as race conditions arising from pipeline behavior or execution-time-dependent control flow on specific microcontrollers [26]. Cycle-accurate instruction set simulators address this by modeling the target pipeline, bus contention, and memory access latencies at clock-cycle granularity. However mature open-source tooling for cycle-accurate simulation is limited [26], they offer limited support for peripherals, and the infrastructure setup overhead is heavier. Our framework's current architecture targets Layers 1 and 2, which cover the functional correctness of a substantial amount of embedded software engineering tasks found in RTOS and firmware repositories. Timing-dependent problems represent a meaningful slice of real-world embedded work, but we defer their inclusion to future iterations of the benchmark.

B. Problem Sourcing Pipeline

Our benchmark problems are derived from real pull requests in actively maintained embedded software repositories. We prioritize repositories that maintain test suites, contain diverse PRs spanning kernel internals, driver implementations, and peripheral abstractions, and target hardware platforms well-supported by our simulation infrastructure. Our problem sourcing initially focuses on the Zephyr RTOS, NuttX, and RIOT OS projects, as they are among the most widely adopted open-source RTOS platforms and satisfy all of these criteria [27] [28] [29]. In the future, we plan to include more popular embedded repositories with sufficient test coverage and compatible build tooling as a problem source.

Fig. 5 illustrates the filtering pipeline used to curate benchmark problems from raw repository data. Starting from the full set of merged pull requests in a target repository, a series of filters is applied to identify suitable problems. First, the pipeline selects PRs that are linked with issues and include associated test modifications, to ensure that each problem has a sufficient description and correctness criterion. Next, it filters for problems that can be evaluated within the simulation infrastructure. Specifically, it searches the test files for mentions of specific platforms allowed to build and run the tests. It also selects PRs with logic-level and driver-level changes in platform-independent parts of the code over issues that require specific physical hardware not modeled by our emulation tools. PRs that involve only documentation changes, CI configuration, or build system fixes are also excluded.

These filters are designed to produce instances that fall within the embedded maintenance workflow described in Section 2. The linked-issue requirement ensures problems are well-scoped tasks, the logic-level and driver-level scope requirement targets the subsystem categories (scheduling, POSIX layers, drivers, and configuration infrastructure) that we identified as underexplored, and the exclusion of documentation and CI-only changes ensures every instance requires modifying existing functional code.

Finally, each candidate that passes filtering is then validated to confirm that it constitutes a well-formed benchmark problem. The validation process runs the PR's tests against two versions of the codebase: first against the parent commit and then against the post-merge code. For a valid benchmark instance, the test should fail on the pre-merge code and pass on the post-merge code.

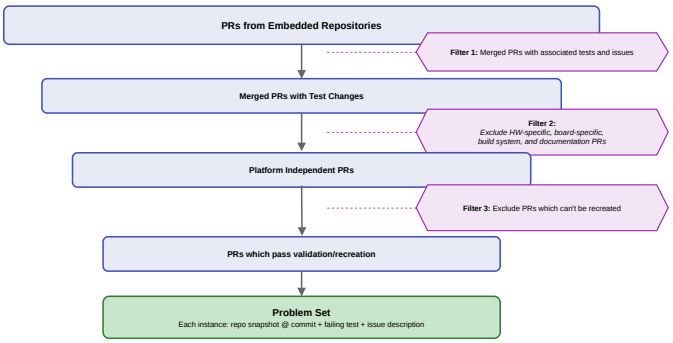

Fig. 5: Problem Sourcing: PR Filtering Pipeline

C. Agent Execution Environment

Each benchmark instance is packaged as a self-contained, reproducible unit built with multi-stage Docker images, following the containerized evaluation methodology used in SWE-bench [9]. A shared base image installs the operating system and the RTOS-specific toolchain required to build and run the firmware. For example, the Zephyr base image installs Ubuntu 24.04 with the west meta-tool and a version-pinned Zephyr SDK; the NuttX base image installs a suite of cross-compilers targeting ARM, AArch64, and RISC-V architectures alongside QEMU; and the RIOT base image installs only the host GCC toolchain, since RIOT's native board target compiles to a host-native binary. Keeping toolchain installation in the base image allows for per-instance images to use a fully configured build environment without repeating expensive setup. Subsequently, per-instance images extend the base by checking out a specific commit of the project repository. Each instance is described by a metadata file that records the base commit (the repository state before the fix), the fail-to-pass tests that must pass after a correct fix, the pass-to-pass tests that must be kept to prevent regression, and the source files the fix should touch. This is adapted from SWE-bench, which uses fail-to-pass and pass-to-pass to assess agent performance at software engineering tasks.

A significant challenge we faced in applying general-purpose coding agents to embedded RTOS codebases is toolchain onboarding. Each project has its own build system, configuration layer, test framework, and execution model that would otherwise consume many agent steps in exploratory commands. To address this, each instance ships a project-level reference document placed inside the container and readable on demand (e.g. Zephyr_guide.md). The document covers the repository layout, build commands, and the project's test framework API. This follows the principle of domain-specific context provisioning described in the SWE-agent Agent-Computer Interface (ACI) framework [30], which demonstrates that equipping agents with domain-relevant context re-

duces wasted actions and improves task completion. Similarly, each instance provides a 'run_tests' script that abstracts the test execution lifecycle (launching the firmware binary/emulator, streaming output, polling for a pass/fail completion string, terminating the process, and exiting with a uniform code: 0 for pass, 1 for fail, and 2 for timeout). Since the underlying execution mechanism varies across targets, this abstraction hides the differences and gives the agent and evaluator a single consistent interface. Surfacing a standard command in place of multi-step invocations is an application of the ACI methodology [30].

D. Agent Harness and Evaluation

We implement the agent harness on top of mini-swe-agent [31], a lightweight open-source framework that provides Docker environment management, per-command timeout enforcement, trajectory and cost logging, and model access via LiteLLM [32]. The LiteLLM abstraction allows the same harness to target any model without modification. Each agent run is configured with a step limit of 50 and a cost ceiling of $3.00 USD per instance. This follows the resource-bounding practice used in SWE-bench evaluations [9] to keep results comparable between models. The agent interacts with the container through two template prompts. A system template establishes the agent's strict response format. Specifically, every reply must contain exactly one bash command block. This constraint is consistent with the structured action space advocated by Yang et al. [30], eases parsing, and prevents the model from issuing multi-step sequences that leave the environment in an undefined intermediate state. An instance template provides the bug description drawn from the original GitHub issue, the fail-to-pass and pass-to-pass test names, and the relevant source file/(s).

Evaluation is deliberately decoupled from agent execution and runs in freshly instantiated containers, separate from the container the agent worked in. For each instance, the evaluator starts a new container from the same instance image, copies the agent's patch file into it, applies the patch with 'git apply', performs a full clean rebuild to discard any cached build state, and runs 'run_tests'. The binary grading marks an instance as pass if 'run_tests' exits with code 0 (i.e. all fail-to-pass tests now pass and all pass-to-pass tests still pass); otherwise it is marked as fail.

IV. Results

A. Problem Set Curation

The final EmbedEval problem set consists of real pull requests from three open-source embedded operating systems: Zephyr, NuttX, and RIOT OS. From a larger pool of candidate PRs, we selected instances with meaningful test coverage and validated each one using the fail-then-pass criterion described earlier. This ensured that every selected task had a concrete test-based correctness signal for evaluating model-generated patches.

The resulting benchmark contains 17 pull request instances: 8 from Zephyr, 6 from NuttX, and 3 from RIOT OS. These instances cover a range of embedded software engineering tasks, including bug fixes, feature additions, kernel or module additions, and refactors. The selected PRs also vary substantially in size. The number of modified files ranges from 1 to 32, while line changes range from a small 7-line modification to a 1031-line kernel-level addition. This variation is important because it allows the benchmark to test both localized fixes and larger changes that require broader repository understanding.

TABLE II: Summary of curated EmbedEval problem set by task type and patch size.

Task Type	**Patch Size (Total Line Diff)**				**Total**
	<100	**100–299**	**300–599**	**≥600**	
Bug fix	7	2	0	0	9
Feature / addition	0	3	1	2	6
Refactor	0	0	1	1	2
Total	**7**	**5**	**2**	**3**	**17**

Table II summarizes the curated problem set by task type and approximate patch size. Patch size is measured using the total number of changed lines, computed as added lines plus removed lines.

Overall, the curated dataset is intentionally small but diverse. It includes both simple bug fixes, which are useful for establishing a baseline level of model competence, and larger feature or refactor tasks, which better test whether models can reason across multiple files and embedded-system abstractions.

B. Overall Model Performance

To evaluate model performance on EmbedEval, we ran our agent loop on all 17 curated PR instances using five frontier or widely used coding models: GPT-5.4, Claude Opus 4.6, Gemini 2.5 Pro, Qwen3-Coder-480B, and DeepSeek-V3. Each model was evaluated on every PR under the same 50-step limit, where one step corresponds to one bash command issued by the model through the agent interface. A run was counted as successful only if the submitted patch, when reapplied in a fresh validation container, caused the previously failing tests to pass.

Table III shows the overall pass rate for each model. GPT-5.4 achieved the highest score, passing 12 of 17 tasks for an overall pass rate of 70.6%. Claude Opus 4.6 followed closely with 11 of 17 tasks passed, or 64.7%. The remaining three models showed a larger drop in performance: Gemini 2.5 Pro passed 8 tasks, while Qwen3-Coder-480B and DeepSeek-V3 each passed 6 tasks.

These results show a clear separation between the two strongest models and the rest of the evaluated set. GPT-5.4 and Claude Opus 4.6 were the only models to solve more than half of the benchmark, while Gemini 2.5 Pro, Qwen3-Coder-480B, and DeepSeek-V3 solved fewer than half of the PRs. At the same time, even the best model reached only 70.6%, leaving substantial room for improvement. This suggests that real embedded software engineering tasks remain challenging for current LLM agents, even when the tasks are test-driven and executed in a controlled environment.

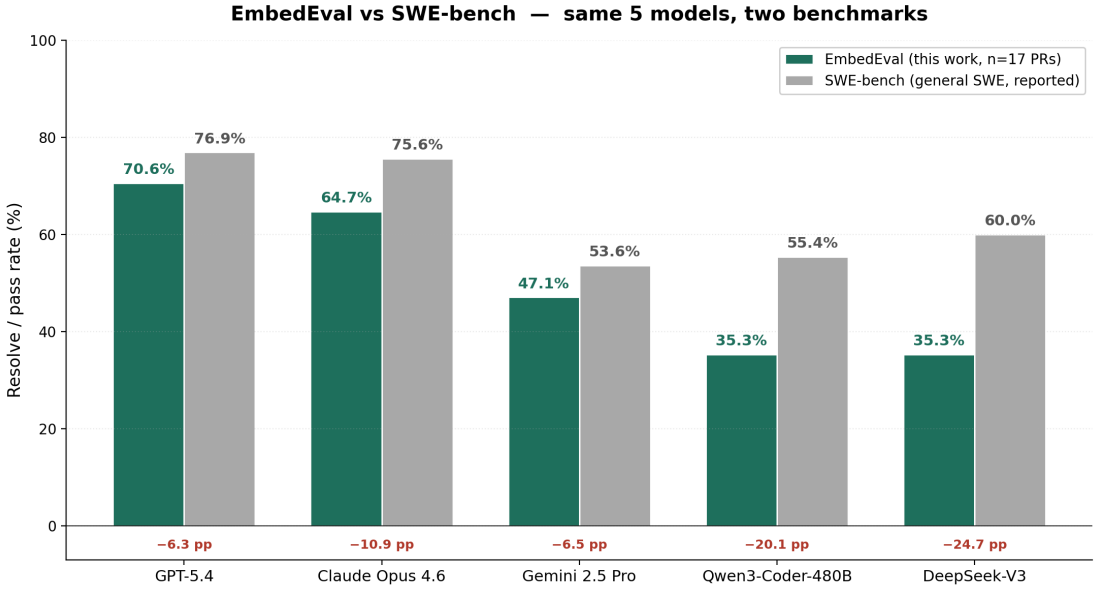

Fig. 6: Comparison of model performance on EmbedEval and reported SWE-bench Verified scores.

TABLE III: Overall EmbedEval performance across 17 PR instances.

Model	Pass	Fail	Pass Rate
GPT-5.4	12	5	70.6%
Claude Opus 4.6	11	6	64.7%
Gemini 2.5 Pro	8	9	47.1%
Qwen3-Coder-480B	6	11	35.3%
DeepSeek-V3	6	11	35.3%

C. Comparison with SWE-Bench

To better contextualize these results, Fig. 6 compares each model's EmbedEval pass rate against its reported SWE-bench Verified score. SWE-bench Verified is a general software engineering benchmark based on real GitHub issues, primarily from Python repositories, and is commonly used to measure repository-level software repair ability [33], [34]. The comparison is not intended as a direct apples-to-apples benchmark equivalence, since EmbedEval is smaller and focuses specifically on embedded RTOS code. However, it is useful for understanding how model performance changes when moving from general software engineering tasks to embedded software engineering tasks.

The comparison shows that every evaluated model performs worse on EmbedEval than on SWE-bench Verified. The drop is relatively modest for the strongest frontier models: GPT-5.4 decreases from 76.9% to 70.6%, Claude Opus 4.6 decreases from 75.6% to 64.7%, and Gemini 2.5 Pro decreases from 53.6% to 47.1%. In contrast, the drop is much larger for Qwen3-Coder-480B and DeepSeek-V3, which fall by roughly 20 to 25 percentage points relative to their SWE-bench comparison scores.

This trend suggests that embedded software tasks expose additional challenges not fully captured by general SWE benchmarks. These include RTOS-specific APIs, build-system constraints, emulator-driven validation, lower-level C code, and changes that may span drivers, kernel components, or platform abstractions. More importantly, the performance gap is not uniform across models: the strongest models remain relatively competitive, while the lower-performing models degrade more sharply. This widens the gap between frontier and open-weight models in the embedded setting and supports the need for a domain-specific benchmark such as EmbedEval.

D. Step Efficiency Comparison

In addition to measuring whether a model successfully solved each PR, we also evaluate how efficiently it reached a passing patch. We measure efficiency using the number of agent steps required for a successful run, where each step corresponds to one bash command issued by the model. Failed runs are excluded from this analysis because they often terminate at or near the 50-step limit and therefore do not provide a meaningful measure of solution efficiency.

To compare efficiency fairly across PRs of different difficulty, we normalize step counts using a per-PR best baseline. For each PR with at least one passing solution, the model that solved that PR in the fewest steps is assigned an efficiency ratio of 1.00. Every other model that also solved the same PR is assigned a ratio equal to its step count divided by the best step count for that PR. We then aggregate these ratios for each model using the geometric mean. Lower values therefore indicate better step efficiency.

Fig. 7 shows a clear efficiency gap between GPT-5.4 and the other evaluated models. GPT-5.4 achieves a geometric mean ratio of 1.00, meaning that it was the most step-efficient model on every PR that it solved. Claude Opus 4.6 is the next most efficient model with a ratio of 2.89, indicating that, on average,

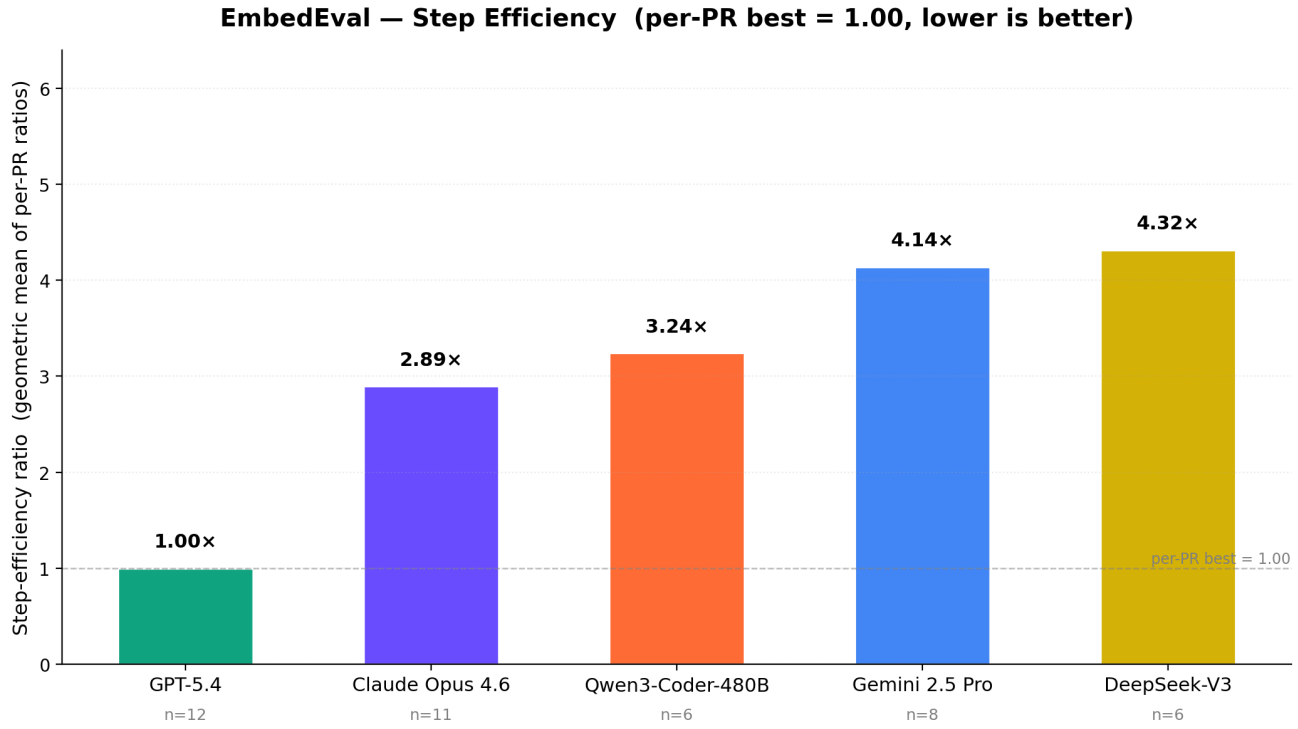

Fig. 7: Step efficiency comparison across models. Each value is the geometric mean of per-PR step ratios over successful runs only. A value of 1.00 represents the most efficient solution for a PR, and lower values indicate better efficiency.

its successful runs required nearly three times as many steps as the per-PR best solution. Qwen3-Coder-480B, Gemini 2.5 Pro, and DeepSeek-V3 required even more steps on successful runs, with ratios of 3.24, 4.14, and 4.32 respectively.

These results are significant because step efficiency reflects more than just speed. In an agentic software engineering setting, fewer steps generally indicate better use of shell feedback, more direct localization of the relevant code, and fewer unnecessary build or debugging attempts. This matters especially for embedded software, where builds and simulator-based tests can be expensive. Fewer steps may also reduce LLM token usage, since each step adds another command, observation, and model response to the interaction history. Therefore, the efficiency results show that model quality affects not only pass rate, but also the practical cost of using LLM agents for embedded software repair.

One caveat is that the number of successful runs differs by model. GPT-5.4 contributes 12 passing runs, while Qwen3-Coder-480B and DeepSeek-V3 contribute only 6 each. As a result, efficiency should be interpreted alongside overall success rate rather than in isolation. PRs that no model solved are also excluded from this analysis. Even with these limitations, the efficiency comparison shows a consistent pattern: the strongest model in pass rate was also the most efficient model in reaching successful solutions.

E. Qualitative Observations

Finally, we note a few recurring patterns observed while inspecting selected PRs and agent trajectories. These observations are not formal metrics, but they help contextualize the quantitative results.

Models often struggled with embedded build-system complexity, especially when a task required coordinating changes across source files, configuration files, and test metadata. We also observed failures related to RTOS-specific semantics, where models appeared to reason about timing, scheduling, or synchronization behavior as if it were ordinary application-level code. Finally, some failures involved low-level embedded programming assumptions, such as bit-level arithmetic, intentional overflow behavior, or hardware-specific conventions.

These observations suggest that EmbedEval tasks require more than general repository-level code repair; they also require awareness of embedded-specific constraints.

V. Statement of Work

A. Devesh Maheshwari

I served as one of the team leads for this project and helped coordinate the team's work throughout the semester. This included planning milestones, delegating tasks, tracking progress, and making sure the team stayed aligned and on schedule.

My technical contributions focused on the design and execution of the benchmark pipeline. I began by conducting a literature review of prior work on LLMs for embedded

software and related benchmarking methods, which helped identify the gap that EmbedEval addresses. After we selected Zephyr RTOS as an initial problem source, I developed an initial testing architecture for identifying pull requests and validating them using the fail-then-pass criterion. I also validated a large number of Zephyr PRs for use in early agent-run demonstrations.

I later shifted focus to NuttX, where I studied its dual-repository structure and created a testing framework for identifying and validating NuttX PR instances. I adapted the existing agent loop for NuttX-specific evaluation, created the required harnesses and `run_tests` scripts for six NuttX PRs, and ran these PRs using all five evaluated models to collect pass/fail and step-count results.

For the final evaluation, I consolidated results across all 17 PR instances from Zephyr, NuttX, and RIOT OS. I created the tables and figures used in the report, compared EmbedEval performance against SWE-bench scores, and analyzed model step efficiency using normalized per-PR step ratios.

My writing and presentation contributions included drafting the initial project abstract, authoring the Related Works section of the initial report, and writing the Results section of the final report. I also presented project progress and results multiple times throughout the semester, including in a research talk.

B. Rishi Mantri

I worked as one of the team leads for the project, helped set the team's technical direction, and coordinated work throughout the semester. This included planning milestones, delegating tasks among team members, and tracking progress against said milestones.

My technical contributions began with the problem sourcing pipeline. I wrote scripts to scrape GitHub for candidate pull requests across embedded RTOS repositories, filtering for PRs that had a linked issue, added tests, and met the fail-to-pass criterion. This ensured instances were verified before being included in the benchmark. Later in the semester, I shifted to working on the execution environment and setting up the multi-level Docker build infrastructure for the Zephyr RTOS problem set. This involved configuring per-instance images with pinned repository commits, and writing `run_tests` scripts that abstracted test execution. I then integrated mini-swe-agent with the Docker environment, wired up the prompt templates, and validated the end-to-end pipeline by running the agent against the containerized instances. This produced the initial set of results on the Zephyr problem set.

On the non technical side, I authored the methodology sections for both the mid-semester and final reports. I also contributed to the presentations and research talk we gave at different points through the semester.

C. Alan Hsi

This semester, I worked on problem sourcing and pull request validation mainly in RIOT. Initially, I validated a few PRs within Zephyr utilizing the twister framework to simulate hardware with QEMU.

Later in the semester, I targeted the RIOT-OS open-source repository in order to expand our benchmark's problem set. Its similarities in testing structure to Zephyr, such as utilizing native simulators, allowed me to easily adjust the pre-existing Zephyr PR validation script to be utilized for RIOT. Furthermore, I modified the agent loop for LLM evaluation on RIOT-specific PRs and developed the necessary harnesses and individual run_test scripts, maintaining consistent structure to the Zephyr pipeline. I benchmarked the five models on each RIOT PR and collected step count, agent exit status, and whether it passed or failed or succeeded the PR.

I authored the future and ongoing work section of the mid-semester report and presented these sections to the class and at the undergraduate research conference.

D. Aanya Mittal

Early in the semester, I participated in the brainstorming of evaluation parameters and testing methodology that could be used to create a benchmark. I then applied these criteria to problem sourcing, identifying and validating pull requests from the NuttX and RIOT OS repositories that met the benchmark's requirements On the infrastructure side, I contributed to the design of the communication layer between the mini swe agent and the Docker execution environment, specifically working out how the mini-swe-agent interacts with the containerized instance through the terminal interface and helping build that pipeline. I also contributed to prompt and token handling within the agent harness. For documentation, I co-authored the introduction and motivation sections of the report and presentation , framing the embedded software market context and the gap that EmbedEval addresses and contributed to the team's presentations throughout the semester.

REFERENCES

[1] "Embedded Software Market Size | Industry Report, 2030," Mar. 2026. [Online]. Available: https://www.grandviewresearch.com/industry-analysis/embedded-software-market-report

[2] "Embedded System Market Size | Industry Report, 2030," Mar. 2026. [Online]. Available: https://www.grandviewresearch.com/industry-analysis/embedded-system-market

[3] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. d. O. Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman, A. Ray, R. Puri, G. Krueger, M. Petrov, H. Khlaaf, G. Sastry, P. Mishkin, B. Chan, S. Gray, N. Ryder, M. Pavlov, A. Power, L. Kaiser, M. Bavarian, C. Winter, P. Tillet, F. P. Such, D. Cummings, M. Plappert, F. Chantzis, E. Barnes, A. Herbert-Voss, W. H. Guss, A. Nichol, A. Paino, N. Tezak, J. Tang, I. Babuschkin, S. Balaji, S. Jain, W. Saunders, C. Hesse, A. N. Carr, J. Leike, J. Achiam, V. Misra, E. Morikawa, A. Radford, M. Knight, M. Brundage, M. Murati, K. Mayer, P. Welinder, B. McGrew, D. Amodei, S. McCandlish, I. Sutskever, and W. Zaremba, "Evaluating Large Language Models Trained on Code," Jul. 2021, arXiv:2107.03374 [cs]. [Online]. Available: http://arxiv.org/abs/2107.03374

[4] J. Austin, A. Odena, M. Nye, M. Bosma, H. Michalewski, D. Dohan, E. Jiang, C. Cai, M. Terry, Q. Le, and C. Sutton, "Program synthesis with large language models," *arXiv preprint arXiv:2108.07732*, 2021.

[5] J. Liu, C. S. Xia, Y. Wang, and L. Zhang, "Is your code generated by chatgpt really correct? rigorous evaluation of large language models for code generation," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[6] Y. Li, D. Choi, J. Chung, N. Kushman, J. Schrittwieser, R. Leblond *et al.*, "Competition-level code generation with alphacode," *Science*, vol. 378, no. 6624, pp. 1092–1097, 2022.

[7] Y. Ding, Z. Wang, W. U. Ahmad, H. Ding, M. Tan, N. Jain, M. K. Ramanathan, R. Nallapati, P. Bhatia, D. Roth, and B. Xiang, "Crosscodeeval: A diverse and multilingual benchmark for cross-file code completion," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[8] N. Jain, K. Han, A. Gu, W.-D. Li, F. Yan, T. Zhang, S. Wang, A. Solar-Lezama, K. Sen, and I. Stoica, "Livecodebench: Holistic and contamination free evaluation of large language models for code," in *International Conference on Machine Learning (ICML)*, 2024.

[9] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan, "SWE-bench: Can Language Models Resolve Real-World GitHub Issues?" Nov. 2024, arXiv:2310.06770 [cs]. [Online]. Available: http://arxiv.org/abs/2310.06770

[10] A. Sonwane, E.-S. Tu, W.-C. Lu, C. Beger, C. Larsen, D. Dhar, S. Alford, R. Chen, R. Pattanayak, T. A. Dang, G. Chen, G. Geng, K. Ellis, and S. Dutta, "OmniCode: A Benchmark for Evaluating Software Engineering Agents," Feb. 2026, arXiv:2602.02262 [cs]. [Online]. Available: http://arxiv.org/abs/2602.02262

[11] S. Haug, C. Böhm, and D. Mayer, "Automated Code Generation and Validation for Software Components of Microcontrollers," Feb. 2025, arXiv:2502.18905 [cs]. [Online]. Available: http://arxiv.org/abs/2502.18905

[12] Yq, K. Gong, Y. Gao, H. Wang, and W. Dong, "IoTMigrator: LLM-driven Embedded IoT Code Migration across Different OSes for Cloud-device Integration," in *Findings of the Association for Computational Linguistics: EMNLP 2025*, C. Christodoulopoulos, T. Chakraborty, C. Rose, and V. Peng, Eds. Suzhou, China: Association for Computational Linguistics, Nov. 2025, pp. 19 245–19 257. [Online]. Available: https://aclanthology.org/2025.findings-emnlp.1048/

[13] K. Gong, W. Dong, H. Wang, Y. Peng, and Y. Gao, "Programming Embedded IoT Applications in Natural Language with IoTPilot," in *Proceedings of the 23rd Annual International Conference on Mobile Systems, Applications and Services*. New York, NY, USA: Association for Computing Machinery, Sep. 2025, pp. 70–82. [Online]. Available: https://dl.acm.org/doi/10.1145/3711875.3729136

[14] Z. Englhardt, R. Li, D. Nissanka, Z. Zhang, G. Narayanswamy, J. Breda, X. Liu, S. Patel, and V. Iyer, "Exploring and Characterizing Large Language Models For Embedded System Development and Debugging," Nov. 2023, arXiv:2307.03817 [cs]. [Online]. Available: http://arxiv.org/abs/2307.03817

[15] R. Xu, J. Cao, M. Wu, W. Zhong, Y. Lu, B. He, X. Han, S.-C. Cheung, and L. Sun, "EmbedAgent: Benchmarking Large Language Models in Embedded System Development," Jan. 2026, arXiv:2506.11003 [cs]. [Online]. Available: http://arxiv.org/abs/2506.11003

[16] F. Speiser, I. Szalay, and D. Fodor, "Embedded System Simulation Using Renode," *Engineering Proceedings*, vol. 79, no. 1, p. 52, 2024. [Online]. Available: https://www.mdpi.com/2673-4591/79/1/52

[17] B. Schirrmeister, F. Geyer, and S. Spathe, "Simulation and Benchmarking of IoT Device Usage Scenarios Using Zephyr and Qemu," 2020.

[18] J. Oliveira, F. Sousa, and L. Almeida, "IoTNetEMU - A Framework to Emulate and Test IoT Applications," in *Proceedings of the 19th ACM International Symposium on QoS and Security for Wireless and Mobile Networks*, ser. Q2SWinet '23. New York, NY, USA: Association for Computing Machinery, Oct. 2023, pp. 33–37. [Online]. Available: https://dl.acm.org/doi/10.1145/3616391.3622774

[19] A. A. Clements, E. Gustafson, T. Scharnowski, P. Grosen, D. Fritz, C. Kruegel, G. Vigna, S. Bagchi, and M. Payer, "HALucinator: firmware re-hosting through abstraction layer emulation," in *Proceedings of the 29th USENIX Conference on Security Symposium*, ser. SEC'20. USA: USENIX Association, Aug. 2020, pp. 1201–1218. [Online]. Available: https://dl.acm.org/doi/10.5555/3489212.3489280

[20] F. N. Presti, L. de Araujo Dantas, L. C. G. de Freitas, M. J. da Cunha, and M. B. de Almeida, "Creating Digital Twins in Renode: A Case Study with STM32L4 and AT86RF233 ZigBee radio," in *2025 16th IEEE International Conference on Industry Applications (INDUSCON)*, Oct. 2025, pp. 677–682, iSSN: 2572-1445. [Online]. Available: https://ieeexplore.ieee.org/document/11241484/

[21] F. Mihalič, M. Truntič, and A. Hren, "Hardware-in-the-Loop Simulations: A Historical Overview of Engineering Challenges," *MDPI*, Jul. 2022. [Online]. Available: https://www.mdpi.com/2079-9292/11/15/2462

[22] "Native simulator - native_sim — Zephyr Project Documentation." [Online]. Available: https://docs.zephyrproject.org/latest/boards/native/native_sim/doc/index.html

[23] "Posix/Linux Simulator Demo for FreeRTOS using GCC - FreeRTOS™." [Online]. Available: https://freertos.org/Documentation/02-Kernel/03-Supported-devices/04-Demos/03-Emulation-and-simulation/Linux/FreeRTOS-simulator-for-Linux

[24] "Welcome to QEMU's documentation! — QEMU documentation." [Online]. Available: https://www.qemu.org/docs/master/index.html

[25] "renode/renode," Mar. 2026, original-date: 2017-06-30T15:52:08Z. [Online]. Available: https://github.com/renode/renode

[26] J. Bauer and F. Freiling, "Towards Cycle-Accurate Emulation of Cortex-M Code to Detect Timing Side Channels," in *2016 11th International Conference on Availability, Reliability and Security (ARES)*, Aug. 2016, pp. 49–58. [Online]. Available: https://ieeexplore.ieee.org/document/7784555

[27] "Zephyr Project Documentation — Zephyr Project Documentation." [Online]. Available: https://docs.zephyrproject.org/latest/

[28] "NuttX documentation," online documentation. [Online]. Available: https://nuttx.apache.org/docs/latest/

[29] "RIOT documentation," online documentation. [Online]. Available: https://guide.riot-os.org/

[30] J. Yang, C. E. Jimenez, A. Wettig, K. Lieret, S. Yao, K. Narasimhan, and O. Press, "SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering," May 2024, arXiv:2405.15793 [cs.SE]. [Online]. Available: http://arxiv.org/abs/2405.15793

[31] K. Lieret, "mini-SWE-agent: Minimal LLM Agents for Software Engineering," 2024, gitHub repository. [Online]. Available: https://github.com/princeton-nlp/mini-swe-agent

[32] BerriAI, "LiteLLM: Call all LLM APIs using the OpenAI format," 2023, gitHub repository. [Online]. Available: https://github.com/BerriAI/litellm

[33] SWE-bench Team, "SWE-bench Official Leaderboards," https://www.swebench.com/, 2026, accessed May 2026.

[34] Epoch AI, "SWE-bench Verified," https://epoch.ai/benchmarks/swe-bench-verified?view=graph&tab=leaderboard, 2026, accessed May 2026.